

Chelsea Chambers

Dr. Lancor

CSC 235-01

Final Project Submission 3

AI-Generated Site Reflection

For my final project, I built MemoBoat, a web app that takes raw meeting notes and uses the OpenAI API to generate a structured memo with a summary, action items, and key decisions. After completing my version, I used Vercel V0 to generate a comparable version using the same specifications and compared the two.

My version follows a straightforward Flask structure. `app.py` handles all the backend logic, HTML templates live in a `/templates` folder, and CSS and JavaScript are in a `/static` folder. Everything is organized the way we learned in class and is easy to follow. V0's structure was more confusing. Even though I specifically told it to use Flask, HTML, Bootstrap, and SQLite only, it still generated extra files like `next.config.mjs`, `tsconfig.json`, and `package.json`, which are Next.js and React files. Those had nothing to do with the project and weren't needed to run the Flask app. It followed the general structure but didn't fully stick to the tech stack I gave it.

Both versions have four pages: Home, Saved Memos, View Memo, and About, with navigation, a generate form, and a database-backed saved memos list. So, the core functionality is similar. The main difference is that my version was built and tested one feature at a time, so I knew exactly why each part worked. V0 generated everything at once, which meant I had to dig into the code to understand what it did before I could even test it.

Both apps use the same three-table SQLite database structure: `meeting_types`, `transcripts`, and `summaries`, with primary keys and foreign keys. V0 actually got this right, which surprised

me. It even pre-generated the memoboat.db file, whereas my version requires running `init_db.py` first to create the database. One difference is that V0 also uses a `base.html` template that all other templates extend from using Jinja2 template inheritance, which is cleaner than what I did. I repeated the nav and footer in every template separately.

The biggest difference was the API call. In my version, I store the API key directly as a string variable in `app.py` and use the `requests` library to call the OpenAI API manually. V0 used `os.environ.get("OPENAI_API_KEY", "")` to read the key from environment variables. It also used the official `openai` Python package instead of `requests`. To get V0's version working locally, I had to change `os.environ.get("OPENAI_API_KEY", "")` to just directly assign my key since I wasn't running it with environment variables configured.

The biggest limitation was that it didn't follow my instructions completely. I explicitly said not to use React or Node, but it generated Next.js config files anyway. The design was also completely generic. It is a purple gradient header and standard card layouts that look like every other AI-generated app. It required the `openai` package which wasn't installed, so it crashed on the first generate attempt until I installed it manually. It also gave no explanation of what it built or why, so understanding the code took extra effort.

Building my version from scratch meant I understood every decision, and when things broke, I knew where to look. V0 is impressive for speed, but it requires you to already know enough to evaluate what it gives you. If I had just submitted the V0 version without understanding it, I wouldn't have been able to explain how any of it worked. The best use of a tool like V0 is probably as a starting point, not a final product.